SkillBarter

Production-Grade Implementation Guide — Peer-to-Peer Skill Exchange Platform

2025–2026

This document is the authoritative build guide for SkillBarter — a peer-to-peer skill exchange platform powered by a token economy. Every section is written to be immediately actionable alongside an AI coding assistant (Cursor, Windsurf, or Claude). Follow phases in order. Do not skip the database schema or LedgerService — they are the foundation everything else depends on.

8	10	11	20
IMPLEMENTATION PHASES	DATABASE TABLES	NESTJS MODULES	FRONTEND ROUTES

Tech Stack Overview

TABLE 1 — FULL TECHNOLOGY STACK

Layer	Technology	Rationale
Frontend	React 19 + TypeScript + Vite + TanStack Query v5 + Shadcn/UI + Tailwind CSS + Zustand	Type-safe, fast HMR, server-state caching, accessible component library
Backend	NestJS (Node.js + TypeScript)	Native TypeScript, modular architecture, built-in WebSocket/Guards/Pipes
Database	PostgreSQL 16	ACID-compliant — mandatory for token ledger integrity, no exceptions
ORM	Prisma	Type-safe queries, excellent DX with NestJS, migration tooling
Cache / Presence	Redis 7	Online status, session caching, rate limiting, Pub/Sub for WebSocket scaling
WebRTC	LiveKit (managed)	Abstracts raw WebRTC

		complexity; provides data channels for sync
File Storage	Cloudinary / S3-compatible	Presigned URL uploads for bounty videos — never route files through NestJS
Auth	JWT (access + refresh) + OAuth2	Stateless access tokens (15m) + httpOnly refresh cookies (7d) + GitHub/LinkedIn
DevOps	Docker + Docker Compose + GitHub Actions + Nginx	Multi-stage builds, automated CI/CD, reverse proxy with WebSocket support

SECTION 1 — OVERALL WEBAPP FLOW

1.1 Platform Philosophy

SkillBarter is built on the Knowledge Time-Bank concept: education is the currency. Rather than charging money for learning, the platform creates a self-sustaining peer-to-peer economy where users teach to earn tokens, then spend tokens to learn. This directly addresses the core problem — financial barriers to upskilling — by replacing monetary transactions with a fair, gamified exchange mechanism.

- Token Economy: SkillCoins (integer, non-divisible) are the sole medium of exchange
  - 50 coins on signup + 50 coins per OAuth verification (GitHub or LinkedIn)
  - Pricing tiers: Beginner (30 min) = 30 coins; Advanced (60 min) = 100 coins
- Async-First Design: Bounty Board enables earning without scheduling live sessions
  - Post a question with a coin reward; experts answer with short video clips
  - Solves cold-start problem — new users can earn immediately
- Escrow-Based Trust: tokens locked at booking, released only on mutual completion
  - Dispute mechanism freezes tokens pending admin review
  - No-show penalty: mentor cancels <1 hour before → loses 10 coins to student
- 5% Transaction Tax: burned to admin wallet on every exchange to prevent inflation
  - Tax formula: Math.floor(amount × 0.05) — always floor division, never round

1.2 The Golden Path User Journey

1. SIGNUP: Email/password registration → bcrypt 12 rounds → 50 SkillCoins credited immediately via LedgerService.credit() → EARN transaction record created
2. OAUTH VERIFICATION: Connect GitHub (pulls repos + contribution graph) → +50 coins if first-time link (idempotent); Connect LinkedIn (pulls job title + education) → +50 coins if first-time link
3. BROWSE MARKETPLACE: Full-text search via PostgreSQL tsvector GIN index → filter by skill category, level (BEGINNER/INTERMEDIATE/EXPERT), price range, 'Available Now' (Redis online status check)
4. BOOK A SESSION: Select mentor skill + time slot → system validates student balance ≥ skill price → LedgerService.lockEscrow() → Escrow record created (LOCKED) → Session record created (PENDING) → Mentor notified via WebSocket event 'session.booked'

5. JOIN LIVE CLASSROOM: 'Join Room' button activates 10 minutes before start_time → LiveKit room token generated on backend → both participants join → session status transitions to ACTIVE
6. LIVE CLASSROOM EXPERIENCE: Split layout — LiveKit video panel (left) + tabbed collaboration panel (right: Monaco Editor for code + Excalidraw whiteboard) + countdown timer synced via LiveKit data channel
7. SESSION COMPLETION: Timer hits 0 OR both click 'Mark Complete' → POST /sessions/:id/complete → when BOTH have marked → LedgerService.releaseEscrow() → 5% tax deducted → net amount credited to mentor → session status = COMPLETED → rating modal appears
8. RATING: POST /reviews {sessionId, rating 1–5, comment} → mentor's reputation_score updated (rolling average) → teaching streak incremented by 1
9. DISPUTE FLOW: Either party clicks 'Raise Dispute' → POST /sessions/:id/dispute → escrow status = DISPUTED → tokens frozen → admin notified → admin reviews → POST /admin/disputes/:id/resolve → tokens released per decision (RELEASE_TO_MENTOR / REFUND_TO_STUDENT / SPLIT)

1.3 Async Bounty Flow

10. User posts bounty: POST /bounties {title, description, rewardCoins} → validate balance ≥ rewardCoins → lock reward in escrow → bounty status = OPEN → expires in 7 days
11. Expert sees bounty on Bounty Board → uploads <2 min video answer via presigned URL (direct to Cloudinary/S3, never through NestJS) → POST /bounties/:id/answers {videoUrl}
12. Asker reviews video answers → POST /bounties/:id/answers/:answerId/accept → LedgerService releases escrow to responder (minus 5% tax) → bounty status = CLOSED
13. Unanswered bounties auto-expire after 7 days via NestJS Scheduler cron job (@Cron EVERY_HOUR) → escrow refunded in full to poster → poster notified

1.4 Tokenomics Engine

TABLE 2 — TOKENOMICS PARAMETERS

Parameter	Value	Notes
Currency	SkillCoins	Integer only, non-divisible, no decimals ever
Signup Bonus	50 coins	Credited immediately on registration — EARN transaction
OAuth Bonus (GitHub)	+50 coins	First-time link only — idempotent, check github_id IS

		NULL before crediting
OAuth Bonus (LinkedIn)	+50 coins	First-time link only — idempotent, check linkedin_id IS NULL before crediting
Beginner Session (30 min)	30 coins	Base pricing tier
Advanced Session (60 min)	100 coins	Base pricing tier
Transaction Tax	Math.floor(amount × 0.05)	Always floor division — burned to admin wallet
No-Show Penalty	min(10, mentorBalance)	Mentor cancels <1 hour before — 10 coins to student
Escrow Lifecycle	LOCKED → RELEASED / DISPUTED / REFUNDED	One-way state transitions only
Admin Wallet	Real user with role=ADMIN	Receives all tax revenue — fully auditable

TABLE 3 — TRANSACTION TYPES

Transaction Type	Description	Direction
EARN	Signup bonus, OAuth bonus, teaching reward	→ User balance
SPEND	Session booking deduction	User balance →
ESCROW_LOCK	Tokens locked at booking	User balance → Escrow
ESCROW_RELEASE	Tokens released on completion	Escrow → Mentor balance
TAX	5% fee on every exchange	Escrow → Admin wallet
PENALTY	No-show deduction from mentor	Mentor → Student
BOUNTY	Bounty reward release	Escrow → Responder balance

1.5 Full Data Flow Architecture

All client traffic enters through Nginx, which routes REST calls to the NestJS API and upgrades WebSocket connections. The NestJS backend is the single source of truth for business logic. Redis handles ephemeral state (online presence, session caching). PostgreSQL via Prisma handles all persistent, ACID-critical data. LiveKit Cloud manages WebRTC rooms independently.

TABLE 4 — SYSTEM ARCHITECTURE LAYERS

Layer	Component	Protocol / Port	Responsibility
Edge	Nginx Reverse Proxy	HTTP :80 / HTTPS :443	TLS termination, gzip, route /api/ and /ws/

API	NestJS API Gateway	HTTP :3000	REST endpoints, Guards, Pipes, Validation
WebSocket	NestJS WS Gateway	WS :3000 /ws	Socket.IO, Redis Pub/Sub broadcast
Database	PostgreSQL 16 (Prisma)	TCP :5432	All persistent data, ACID transactions
Cache	Redis 7	TCP :6379	Online status (TTL 30s), session cache, rate limiting
WebRTC	LiveKit Cloud	HTTPS / WSS	Video/audio rooms, data channels for sync
Storage	Cloudinary / S3	HTTPS	Bounty video uploads via presigned URLs
Frontend	React 19 + Vite	HTTP :5173 (dev)	SPA, TanStack Query, Zustand, Shadcn/UI

1.6 Complete Database Schema

> CRITICAL: Use PostgreSQL for ALL data. Never use MongoDB for a token economy. Every token movement must be ACID-compliant. The transactions table is the immutable audit log — never delete records from it.

Table: users

TABLE 5 — USERS SCHEMA

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt 12 rounds — never expose in API responses
token_balance	INTEGER	NOT NULL, DEFAULT 50, CHECK >= 0	Current SkillCoin balance — never goes negative
reputation_score	DECIMAL(3,2)	DEFAULT 0.00	Rolling average of received ratings (0.00–5.00)
is_verified	BOOLEAN	DEFAULT false	True after phone or OAuth verification
github_id	VARCHAR(100)	UNIQUE, NULLABLE	GitHub OAuth ID — used for idempotent bonus

			check
linkedin_id	VARCHAR(100)	UNIQUE, NULLABLE	LinkedIn OAuth ID — used for idempotent bonus check
teaching_streak	INTEGER	DEFAULT 0	Consecutive sessions taught — incremented on escrow release
role	ENUM	USER / ADMIN, DEFAULT USER	Role-based access control
is_banned	BOOLEAN	DEFAULT false	Banned users cannot book or teach
created_at	TIMESTAMPTZ	DEFAULT NOW()	Account creation timestamp
updated_at	TIMESTAMPTZ	AUTO-UPDATE	Last profile update

Table: skills

TABLE 6 — SKILLS SCHEMA

Column	Type	Constraints	Description
id	UUID	PK	Primary key
user_id	UUID	FK → users.id, NOT NULL	Skill owner (mentor)
title	VARCHAR(100)	NOT NULL	Skill title — included in FTS vector
description	TEXT	NOT NULL	Full description — included in FTS vector
category	VARCHAR(50)	NOT NULL	e.g. 'Web Development', 'Design', 'Data Science'
level	ENUM	BEGINNER / INTERMEDIATE / EXPERT	Skill difficulty level
price_coins	INTEGER	NOT NULL, CHECK > 0	Cost in SkillCoins per session
is_active	BOOLEAN	DEFAULT true	Soft delete — false hides from marketplace
search_vector	TSVECTOR	GIN INDEX	to_tsvector('english', title || ' ' || description || ' ' || category)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp

Table: sessions

TABLE 7 — SESSIONS SCHEMA

Column	Type	Constraints	Description
id	UUID	PK	Primary key
mentor_id	UUID	FK → users.id, NOT NULL	The teaching user
student_id	UUID	FK → users.id, NOT NULL	The learning user
skill_id	UUID	FK → skills.id, NOT NULL	The skill being taught
start_time	TIMESTAMPTZ	NOT NULL	Scheduled start — must be in the future at booking
end_time	TIMESTAMPTZ	NOT NULL	Calculated: start + 30min (BEGINNER) or 60min (EXPERT)
livekit_room_id	VARCHAR(100)	NULLABLE	Generated when first participant joins
status	ENUM	PENDING/ACTIVE/COMPLETED/DISPUTED/CANCELLED, DEFAULT PENDING	Session lifecycle state
mentor_completed	BOOLEAN	DEFAULT false	Mentor clicked 'Mark Complete'
student_completed	BOOLEAN	DEFAULT false	Student clicked 'Mark Complete'
created_at	TIMESTAMPTZ	DEFAULT NOW()	Booking timestamp

Table: transactions

TABLE 8 — TRANSACTIONS SCHEMA (IMMUTABLE AUDIT LOG)

Column	Type	Constraints	Description
id	UUID	PK	Primary key
from_user_id	UUID	FK → users.id, NULLABLE	Sender — NULL for EARN (system credit)
to_user_id	UUID	FK → users.id, NULLABLE	Receiver — NULL for TAX burn
amount	INTEGER	NOT NULL, CHECK > 0	Coin amount — always positive
type	ENUM	EARN/SPEND/ESCROW_LOCK/ESCROW_RELEASE/TAX/PENALTY/BOUNTY	Transaction classification
status	ENUM	PENDING/COMPLETED/FAILED, DEFAULT COMPLETED	Transaction status
reference_id	UUID	NULLABLE	Links to session_id, bounty_id, or escrow_id
created_at	TIMESTAMPTZ	DEFAULT NOW()	Immutable audit timestamp — never update this table

Table: escrows

TABLE 9 — ESCROWS SCHEMA

Column	Type	Constraints	Description
id	UUID	PK	Primary key
session_id	UUID	FK → sessions.id, UNIQUE, NULLABLE	Linked session — one escrow per

			session
bounty_id	UUID	FK → bounties.id, UNIQUE, NULLABLE	Linked bounty — one escrow per bounty
amount	INTEGER	NOT NULL, CHECK > 0	Locked coin amount
status	ENUM	LOCKED/RELEASED/DISPUTED/REFUNDED, DEFAULT LOCKED	Escrow lifecycle — one-way transitions
created_at	TIMESTAMPTZ	DEFAULT NOW()	Lock timestamp
resolved_at	TIMESTAMPTZ	NULLABLE	Set when status changes from LOCKED

Tables: bounties, bounty_answers, reviews, notifications, disputes

TABLE 10 — SUPPORTING TABLES SUMMARY

Table	Key Columns	Critical Constraints
bounties	id, poster_id FK, title, description, category, reward_coins, status ENUM(OPEN/ANSWERED/CLOSED/EXPIRED), expires_at	reward_coins CHECK > 0; expires_at DEFAULT NOW()+7days; status DEFAULT OPEN
bounty_answers	id, bounty_id FK, responder_id FK, video_url, is_accepted DEFAULT false, created_at	Only one answer per bounty can have is_accepted=true; validate video_url domain
reviews	id, session_id FK, reviewer_id FK, reviewee_id FK, rating, comment NULLABLE, created_at	rating CHECK BETWEEN 1 AND 5; one review per session per reviewer
notifications	id, user_id FK, type, message, is_read DEFAULT false, created_at	Always create DB record BEFORE emitting WebSocket event
disputes	id, session_id FK UNIQUE, raised_by FK, reason, status ENUM(OPEN/RESOLVED), admin_notes NULLABLE, decision ENUM(RELEASE_TO_MENTOR/REFUND_TO_STUDENT/SPLIT) NULLABLE, created_at, resolved_at NULLABLE	admin_notes required on resolution (min 20 chars); decision required on resolution

1.7 NestJS Module Map

TABLE 11 — NESTJS MODULE MAP

Module	Responsibilities	Key Services / Controllers
AuthModule	Registration, login, JWT access+refresh tokens, GitHub OAuth, LinkedIn OAuth, signup/OAuth bonuses	AuthService, AuthController, JwtStrategy, GithubStrategy, LinkedinStrategy
UsersModule	User profiles, availability slots, online status, profile updates	UsersService, UsersController, AvailabilityService
LedgerModule	ALL token operations — credit, debit, lockEscrow, releaseEscrow, refundEscrow, freezeEscrow, penalties, bounty escrows	LedgerService (ACID-critical), WalletController, TransactionsController
SkillsModule	Skill CRUD, ownership validation, FTS search index maintenance, soft delete	SkillsService, SkillsController
SessionsModule	Booking flow, overlap validation, LiveKit token generation, session lifecycle, completion/dispute flows	SessionsService, SessionsController, LiveKitService
BountiesModule	Bounty CRUD, answer submission, accept answer, expiry cron job	BountiesService, BountiesController, BountyExpiryScheduler
NotificationsModule	DB-persisted notifications, real-time delivery via WsGateway, read/unread management	NotificationsService, NotificationsController
ReviewsModule	Session reviews, reputation score recalculation (rolling average)	ReviewsService, ReviewsController
DisputesModule	Dispute creation, admin resolution with three decision types, audit trail	DisputesService (used by AdminModule)
AdminModule	All admin routes (ADMIN role guard), dispute resolution, user ban/unban, platform stats, admin wallet	AdminController, AdminStatsService
UploadModule	Presigned URL generation for Cloudinary/S3 — video uploads never route through NestJS	UploadService, UploadController
HealthModule	Liveness + readiness probes for Docker HEALTHCHECK and load balancer	HealthController (Terminus: Prisma + Redis indicators)

1.8 Frontend Route Map

TABLE 12 — FRONTEND ROUTE MAP (20 ROUTES)

Route	Component	Auth Required	Description
/	LandingPage	No	Marketing page with hero, features, CTA
/register	RegisterPage	No (redirect if authed)	Email/password registration form
/login	LoginPage	No (redirect if authed)	Login form + GitHub/LinkedIn OAuth buttons
/auth/callback	OAuthCallbackPage	No	Handles OAuth redirect, stores tokens, redirects to /dashboard
/dashboard	DashboardPage	Yes	User home: balance card, upcoming sessions, recent activity
/marketplace	MarketplacePage	No	Skill search with filters, skill cards, booking entry point
/marketplace/:skillId	SkillDetailPage	No	Full skill info, mentor profile, reviews, booking section
/skills/new	CreateSkillPage	Yes	Form to add a new skill listing
/skills/:id/edit	EditSkillPage	Yes (owner only)	Edit existing skill listing
/sessions	SessionsListPage	Yes	All user sessions (upcoming, past, disputed)
/sessions/:id	SessionDetailPage	Yes	Session info, status, join button, dispute button
/classroom/:sessionId	LiveClassroomPage	Yes (participant only)	Full-screen: LiveKit video + Monaco + Excalidraw + timer (lazy loaded)
/bounties	BountyBoardPage	No	Bounty listing with filters, post bounty button
/bounties/:id	BountyDetailPage	No (post answer requires auth)	Full bounty, video answers, accept button for poster
/wallet	WalletPage	Yes	Balance card, summary stats, transaction history table

/profile/:userId	PublicProfilePage	No	Public profile: skills, reviews, GitHub repos, streak
/profile/me	MyProfilePage	Yes	Edit profile, manage skills, availability settings
/notifications	NotificationsPage	Yes	Full notification history with read/unread management
/admin	AdminDashboardPage	Yes (ADMIN)	Stats cards, charts (signups, sessions, categories)
/admin/disputes	AdminDisputesPage	Yes (ADMIN)	Disputes table, detail modal, resolve form
/admin/users	AdminUsersPage	Yes (ADMIN)	User search, ban/unban actions
/admin/wallet	AdminWalletPage	Yes (ADMIN)	Tax revenue balance, charts, transaction history

SECTION 2 — PHASE-WISE IMPLEMENTATION PROMPTS

Each phase below contains an overview, key deliverables, and a complete implementation prompt ready to paste into Cursor, Windsurf, or Claude. Execute phases in order — each phase depends on the previous. The prompts are written to be self-contained and precise enough for an AI coding assistant to produce working code on the first attempt.

> WORKFLOW: Copy the implementation prompt for each phase into your AI coding assistant. Review the generated code against the critical rules listed in each phase before moving to the next phase.

Phase 1 — Project Setup + Auth + DB Schema

This phase establishes the entire project foundation — monorepo structure, database schema, and authentication system. Getting auth and schema right first prevents costly refactors later. The JWT + OAuth2 flow with signup bonuses establishes the token economy from day one. Every subsequent phase depends on the Prisma schema defined here.

Key Deliverables

- Monorepo with /apps/frontend and /apps/backend (pnpm workspaces)
- Complete Prisma schema with all 10 tables and enums, migrations applied
- NestJS AuthModule: email/password + JWT access/refresh + GitHub OAuth + LinkedIn OAuth
- Signup bonus (50 coins) and OAuth verification bonus (+50 coins each) via LedgerService
- Frontend: Register, Login, OAuth callback pages + Zustand auth store + ProtectedRoute HOC
- Seed script: admin user + 5 sample skills + 2 regular users with token balances

Implementation Prompt — Phase 1

Paste the following prompt into your AI coding assistant (Cursor / Windsurf / Claude):

PHASE 1 PROMPT — PROJECT SETUP + AUTH + DB SCHEMA

Build the complete foundation for SkillBarter, a peer-to-peer skill exchange platform with a token economy.

== MONOREPO STRUCTURE ==
Create a pnpm workspaces monorepo:
  /apps/backend — NestJS application
  /apps/frontend — React 19 + Vite application
  /packages/shared — shared TypeScript types

pnpm-workspace.yaml at root
Root package.json with scripts: dev, build, lint

== PRISMA SCHEMA (apps/backend/prisma/schema.prisma) ==
Create ALL enums and models exactly as specified:

Enums: Role (USER, ADMIN), SkillLevel (BEGINNER, INTERMEDIATE, EXPERT), SessionStatus (PENDING, ACTIVE, COMPLETED, DISPUTED, CANCELLED), TransactionType (EARN, SPEND, ESCROW_LOCK, ESCROW_RELEASE, TAX, PENALTY, BOUNTY), TransactionStatus (PENDING, COMPLETED, FAILED), EscrowStatus (LOCKED, RELEASED, DISPUTED, REFUNDED), BountyStatus (OPEN, ANSWERED, CLOSED, EXPIRED), DisputeDecision (RELEASE_TO_MENTOR, REFUND_TO_STUDENT, SPLIT)

Models: User, Skill, Session, Transaction, Escrow, Bounty, BountyAnswer, Review, Notification, Dispute — with all columns, constraints, and relations as specified in Section 1.6 of the build guide.

Critical schema rules:
- token_balance: Int @default(50) with CHECK constraint >= 0
- search_vector on Skill: Unsupported('tsvector')? — handle via raw SQL migration
- All FKs with proper onDelete behavior (Restrict for financial tables, Cascade for notifications)
- Add @@index on sessions(mentorId), sessions(studentId), transactions(fromUserId), transactions(toUserId), transactions(referenceId)

After schema: run 'prisma migrate dev --name init'

== NESTJS BACKEND SETUP ==
Install: @nestjs/passport, passport, passport-jwt, passport-github2, passport-linkedin-oauth2, @nestjs/jwt, bcrypt, @types/bcrypt, @prisma/client, prisma, ioredis, @nestjs/config, class-validator, class-transformer, @nestjs/throttler

Create PrismaService (extends PrismaClient, implements OnModuleInit/OnModuleDestroy)
Create RedisService (ioredis client, onModuleDestroy disconnect)

== LEDGER SERVICE (apps/backend/src/ledger/ledger.service.ts) ==
Implement ONLY the credit() method in this phase (others in Phase 2):
  async credit(userId: string, amount: number, type: TransactionType, referenceId?: string): Promise<void>

— Use prisma.$transaction([user.update increment tokenBalance, transaction.create])
— Throw if amount <= 0

== AUTH MODULE ==
AuthService methods:
register(dto: RegisterDto): hash password with bcrypt rounds=12, create user (tokenBalance starts at 50 per schema default), call ledger.credit(userId, 50, EARN) to create the transaction record, return { user, accessToken, refreshToken }
login(dto: LoginDto): validate credentials, generateTokens
generateTokens(userId, role): accessToken (JWT, 15m, payload: {sub, role}), refreshToken (JWT, 7d, payload: {sub})
refreshTokens(refreshToken): verify, generate new pair
logout(userId): invalidate refresh token (store hash in Redis with TTL 7d)
githubCallback(profile): find or create user by github_id, if NEW github link (github_id was null): call ledger.credit(userId, 50, EARN) — IDEMPOTENT check: only credit if user.githubId was null before this call
linkedinCallback(profile): same pattern as github

JWT Strategy: extract Bearer token from Authorization header, validate sub exists in DB, attach full user to request
JwtAuthGuard: extends AuthGuard('jwt')
RolesGuard: checks request.user.role against @Roles() decorator metadata

GitHub OAuth Strategy (passport-github2): clientID/Secret from env, callbackURL /auth/github/callback, scope ['user:email', 'read:user']
LinkedIn OAuth Strategy: clientID/Secret from env, callbackURL /auth/linkedin/callback, scope ['r_emailaddress', 'r_liteprofile']

Auth Controller endpoints:
POST /auth/register — @Body RegisterDto (email, password min 8 chars, name)
POST /auth/login — @Body LoginDto, set refreshToken as httpOnly cookie
POST /auth/refresh — read refreshToken from cookie, return new accessToken
POST /auth/logout — @UseGuards(JwtAuthGuard), clear cookie, invalidate in Redis
GET /auth/github — @UseGuards(AuthGuard('github'))
GET /auth/github/callback — @UseGuards(AuthGuard('github')), redirect to frontend /auth/callback?token=...
GET /auth/linkedin — @UseGuards(AuthGuard('linkedin'))

GET /auth/linkedin/callback — @UseGuards(AuthGuard('linkedin')), redirect to frontend /auth/callback?token=...

== SEED SCRIPT (apps/backend/prisma/seed.ts) ==
Create:
1 admin user: email=admin@skillbarter.dev, role=ADMIN, tokenBalance=999999
2 regular users with tokenBalance=150 each
5 skills across different categories and levels
Run with: prisma db seed

== FRONTEND SETUP ==
Install: react-router-dom v6, @tanstack/react-query v5, zustand, axios, shadcn/ui (init), tailwindcss

Zustand AuthStore (src/stores/authStore.ts):
interface AuthState { user: User | null; accessToken: string | null; isAuthenticated: boolean; setAuth(user, accessToken): void; clearAuth(): void; updateBalance(newBalance: number): void }
CRITICAL: accessToken stored in memory ONLY (Zustand state, NOT localStorage)
Persist ONLY { user } to localStorage (not the token)
On app load: if user in localStorage, call POST /auth/refresh to get new accessToken silently

Axios instance (src/lib/axios.ts):
baseURL from VITE_API_URL env
Request interceptor: attach accessToken from Zustand store to Authorization header
Response interceptor: on 401, call POST /auth/refresh, retry original request once, on second 401 call clearAuth()

Pages:
RegisterPage: form with name/email/password, call POST /auth/register, on success setAuth + navigate /dashboard
LoginPage: form with email/password + GitHub/LinkedIn OAuth buttons (links to /api/auth/github), on success setAuth + navigate /dashboard
OAuthCallbackPage: read ?token= from URL params, call GET /auth/me with token, setAuth, navigate /dashboard

ProtectedRoute HOC: if !isAuthenticated redirect to /login with ?redirect= param

== .env.example ==

Include ALL variables:
DATABASE_URL, REDIS_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, GITHUB_CLIENT_ID, GITHUB_CLIENT_SECRET, GITHUB_CALLBACK_URL, LINKEDIN_CLIENT_ID, LINKEDIN_CLIENT_SECRET, LINKEDIN_CALLBACK_URL, FRONTEND_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, LIVEKIT_URL, CLOUDINARY_URL (or S3 vars), ADMIN_EMAIL

== CRITICAL RULES ==
- Access token NEVER in localStorage — memory only
- Refresh token ONLY in httpOnly cookie
- bcrypt MUST be 12 rounds
- Signup bonus MUST create a Transaction record (not just increment balance)
- OAuth bonus is IDEMPOTENT — check github_id/linkedin_id was null before crediting
- NEVER return passwordHash in any API response (use class-transformer @Exclude)
- All DTOs use class-validator decorators
- Enable ValidationPipe globally with whitelist: true, forbidNonWhitelisted: true, transform: true

Phase 2 — Token Economy + Escrow Logic (LedgerService)

This phase implements the financial heart of SkillBarter — the LedgerService. Every token movement must be ACID-compliant using Prisma transactions to prevent double-spends, race conditions, and balance corruption. This is the most critical service in the entire platform. A bug here means real token loss.

Key Deliverables

- LedgerService with all 8 token operations wrapped in Prisma transactions
- EscrowService with full lifecycle management (LOCK → RELEASE / DISPUTE / REFUND)
- Tax calculation logic (5% floor division, admin wallet as real user)
- No-show penalty logic with balance floor protection
- Wallet API endpoints: balance, summary, paginated transaction history
- Frontend: WalletPage with animated balance card, summary stats, filterable transaction table

Implementation Prompt — Phase 2

PHASE 2 PROMPT — TOKEN ECONOMY + ESCROW LOGIC

Implement the complete LedgerService for SkillBarter. This is the most critical service — every method MUST use prisma.$transaction() for atomicity.

```
== LEDGER SERVICE (Complete Implementation) ==

async credit(userId: string, amount: number, type: TransactionType, referenceId?: string):
Promise<void>
  Validate: amount must be > 0, throw BadRequestException otherwise
  prisma.$transaction([
    prisma.user.update({ where: {id: userId}, data: { tokenBalance: { increment: amount } } }),
    prisma.transaction.create({ data: { toUserId: userId, amount, type, status: COMPLETED,
referenceId } })
  ])

async debit(userId: string, amount: number, type: TransactionType, referenceId?: string):
Promise<void>
  First: const user = await prisma.user.findUniqueOrThrow({ where: {id: userId} })
  Check: if (user.tokenBalance < amount) throw new BadRequestException('Insufficient balance')
  prisma.$transaction([
    prisma.user.update({ where: {id: userId}, data: { tokenBalance: { decrement: amount } } }),
    prisma.transaction.create({ data: { fromUserId: userId, amount, type: SPEND, status:
COMPLETED, referenceId } })
  ])

async lockEscrow(sessionId: string, studentId: string, amount: number): Promise<Escrow>
  Validate balance first (same check as debit)
  prisma.$transaction(async (tx) => {
    await tx.user.update({ where: {id: studentId}, data: { tokenBalance: { decrement: amount } } })
    await tx.transaction.create({ data: { fromUserId: studentId, amount, type: ESCROW_LOCK,
referenceId: sessionId } })
    return tx.escrow.create({ data: { sessionId, amount, status: LOCKED } })
  })

async releaseEscrow(sessionId: string): Promise<void>
  Find escrow by sessionId, validate status === LOCKED
  Find session to get mentorId
  const tax = Math.floor(escrow.amount * 0.05)  // ALWAYS Math.floor, never Math.round
  const netToMentor = escrow.amount - tax
  const adminUser = await prisma.user.findFirst({ where: { role: ADMIN } })
  prisma.$transaction(async (tx) => {
```

```
    await tx.user.update({ where: {id: mentorId}, data: { tokenBalance: { increment: netToMentor },
teachingStreak: { increment: 1 } } })
    await tx.user.update({ where: {id: adminUser.id}, data: { tokenBalance: { increment: tax } } })
    await tx.transaction.create({ data: { fromUserId: null, toUserId: mentorId, amount: netToMentor,
type: ESCROW_RELEASE, referenceId: sessionId } })
    await tx.transaction.create({ data: { fromUserId: null, toUserId: adminUser.id, amount: tax, type:
TAX, referenceId: sessionId } })
    await tx.escrow.update({ where: {sessionId}, data: { status: RELEASED, resolvedAt: new Date() } })
  })

async refundEscrow(sessionId: string): Promise<void>
  Find escrow, validate status === LOCKED (cannot refund RELEASED or DISPUTED)
  Find session to get studentId
  prisma.$transaction(async (tx) => {
    await tx.user.update({ where: {id: studentId}, data: { tokenBalance: { increment: escrow.amount }
} })
    await tx.transaction.create({ data: { toUserId: studentId, amount: escrow.amount, type:
ESCROW_RELEASE, referenceId: sessionId } })
    await tx.escrow.update({ where: {sessionId}, data: { status: REFUNDED, resolvedAt: new Date() }
})
  })

async freezeEscrow(sessionId: string): Promise<void>
  Find escrow, validate status === LOCKED
  prisma.escrow.update({ where: {sessionId}, data: { status: DISPUTED } })
  // Note: resolvedAt is NOT set here — set only when admin resolves

async applyNoShowPenalty(mentorId: string, studentId: string, sessionId: string): Promise<void>
  const PENALTY_AMOUNT = 10
  const mentor = await prisma.user.findUniqueOrThrow({ where: {id: mentorId} })
  const penaltyAmount = Math.min(PENALTY_AMOUNT, mentor.tokenBalance)  // Cannot go below
0
  if (penaltyAmount === 0) return  // Mentor is broke, skip penalty
  prisma.$transaction([
    prisma.user.update({ where: {id: mentorId}, data: { tokenBalance: { decrement: penaltyAmount }
} }),
    prisma.user.update({ where: {id: studentId}, data: { tokenBalance: { increment: penaltyAmount } }
```

```
}),
    prisma.transaction.create({ data: { fromUserId: mentorId, toUserId: studentId, amount:
penaltyAmount, type: PENALTY, referenceId: sessionId } })
  ])

async lockBountyEscrow(bountyId: string, posterId: string, amount: number): Promise<Escrow>
  Same as lockEscrow but use bountyId instead of sessionId
  prisma.$transaction: debit poster + ESCROW_LOCK transaction + escrow.create({ bountyId,
amount, status: LOCKED })

async releaseBountyEscrow(bountyId: string, responderId: string): Promise<void>
  Same as releaseEscrow but find escrow by bountyId, credit responderId
  tax = Math.floor(amount * 0.05), net = amount - tax, credit responder + admin

== WALLET CONTROLLER ==
All routes: @UseGuards(JwtAuthGuard)

GET /wallet/balance
  Return: { balance: user.tokenBalance, teachingStreak: user.teachingStreak, reputationScore:
user.reputationScore }

GET /wallet/summary
  Aggregate from transactions table for current user:
  totalEarned: SUM of EARN + ESCROW_RELEASE + BOUNTY transactions where toUserId = me
  totalSpent: SUM of SPEND + ESCROW_LOCK transactions where fromUserId = me
  totalSessions: COUNT of sessions where mentorId = me AND status = COMPLETED
  totalBountiesAnswered: COUNT of bountyAnswers where responderId = me AND isAccepted =
true
  Return all four values

GET /transactions/me
  Query params: page (default 1), limit (default 20, max 100), type (optional filter)
  WHERE: fromUserId = me OR toUserId = me
  ORDER BY: createdAt DESC
  Return: { data: Transaction[], total, page, limit }

== FRONTEND WALLET PAGE ==
```

WalletPage (/wallet):

BalanceCard component:
  Large animated number display (framer-motion count-up on mount)
  Shows: current balance, teaching streak with fire emoji, reputation score with stars
  Shadcn Card with purple gradient background

SummaryCards (3 cards in a row):
  Total Earned (green), Total Spent (red), Sessions Completed (blue)
  Use TanStack Query: useQuery({ queryKey: ['wallet', 'summary'], queryFn: getWalletSummary, staleTime: 30_000 })

TransactionHistoryTable:
  Columns: Date, Type (colored badge), Amount (+/-), Description (reference)
  Type badge colors: EARN=green, SPEND=red, ESCROW_LOCK=yellow, ESCROW_RELEASE=green, TAX=gray, PENALTY=orange, BOUNTY=purple
  Pagination: useInfiniteQuery or manual page state
  Filter dropdown: All Types | EARN | SPEND | ESCROW | PENALTY | BOUNTY
  Use Shadcn Table + Badge components

== CRITICAL RULES ==
- NEVER update tokenBalance without creating a Transaction record in the same prisma.$transaction
- ALWAYS use prisma.$transaction() — never sequential awaits for financial operations
- Tax is ALWAYS Math.floor() — never Math.round() or Math.ceil()
- tokenBalance MUST NEVER go below 0 — check before every debit
- Escrow status transitions are ONE-WAY: LOCKED can go to RELEASED, DISPUTED, or REFUNDED — never backwards
- The admin wallet is a REAL user in the database — not a magic number
- releaseEscrow increments teachingStreak in the SAME transaction as the credit
- freezeEscrow does NOT set resolvedAt — only admin resolution sets resolvedAt

Phase 3 — Marketplace + Search + Matching

This phase builds the discovery layer — how students find mentors and book sessions. The marketplace is the primary user-facing feature and must feel fast and responsive. PostgreSQL Full Text Search with a GIN index provides sub-100ms search without the operational overhead of

Elasticsearch. The booking flow is atomic — escrow lock and session creation happen in a single Prisma transaction.

Key Deliverables

- Skills CRUD API with ownership validation and soft delete
- Marketplace search with PostgreSQL FTS (tsvector GIN index), filters, and dynamic sorting
- User availability slots system (day-of-week + hour ranges)
- Booking flow: 5 validations → balance check → escrow lock → session creation → mentor notification
- Frontend: MarketplacePage with debounced search, FilterSidebar, SkillCards
- BookingModal with slot picker, balance display, and escrow warning
- SkillDetailPage with mentor profile, GitHub repos, reviews, and booking section

Implementation Prompt — Phase 3

```
PHASE 3 PROMPT — MARKETPLACE + SEARCH + MATCHING

Build the skill marketplace and session booking system for SkillBarter.

== SKILLS CRUD ==
POST /skills — @UseGuards(JwtAuthGuard)
  DTO: CreateSkillDto { title: string (max 100), description: string, category: string, level: SkillLevel,
priceCoins: number (min 1) }
  After create: run raw SQL to update search_vector:
    UPDATE skills SET search_vector = to_tsvector('english', title || ' ' || description || ' ' || category)
WHERE id = $1
  Return created skill

GET /skills/:id — public
  Include: user (name, reputationScore, teachingStreak, githubId), reviews (last 5)
  NEVER include passwordHash

PUT /skills/:id — @UseGuards(JwtAuthGuard)
  Ownership guard: if skill.userId !== req.user.id throw ForbiddenException
  Update fields + regenerate search_vector

DELETE /skills/:id — @UseGuards(JwtAuthGuard)
```

```
  Ownership guard
  Soft delete: prisma.skill.update({ data: { isActive: false } })
  Do NOT hard delete — sessions may reference this skill

== FULL TEXT SEARCH SETUP ==
In Prisma migration, add raw SQL:
  CREATE INDEX skills_search_idx ON skills USING GIN(search_vector);
  CREATE OR REPLACE FUNCTION skills_search_vector_update() RETURNS trigger AS $$
  BEGIN
    NEW.search_vector := to_tsvector('english', NEW.title || ' ' || NEW.description || ' ' ||
NEW.category);
    RETURN NEW;
  END;
  $$ LANGUAGE plpgsql;
  CREATE TRIGGER skills_search_vector_trigger BEFORE INSERT OR UPDATE ON skills
    FOR EACH ROW EXECUTE FUNCTION skills_search_vector_update();

== MARKETPLACE SEARCH ENDPOINT ==
GET /marketplace/search — public
Query params:
  q: string (optional) — full text search query
  category: string (optional)
  level: SkillLevel (optional)
  minPrice: number (optional)
  maxPrice: number (optional)
  availableNow: boolean (optional) — check Redis for online status
  sortBy: 'rating' | 'price_asc' | 'price_desc' | 'newest' | 'streak' (default: 'rating')
  page: number (default 1)
  limit: number (default 12, max 50)

Search logic:
  If q provided: WHERE search_vector @@ plainto_tsquery('english', q)
  Fallback if no FTS results: WHERE title ILIKE '%q%' OR description ILIKE '%q%'
  Always add: WHERE isActive = true
  If availableNow: filter userIds by checking Redis keys user:{userId}:online
  Sort mapping: rating → user.reputationScore DESC, price_asc → priceCoins ASC, price_desc →
priceCoins DESC, newest → createdAt DESC, streak → user.teachingStreak DESC
```

```
  Include: user (id, name, reputationScore, teachingStreak, githubId, isVerified)
  Return: { data: SkillWithUser[], total, page, limit, hasMore }

== USER AVAILABILITY ==
Add UserAvailability model to Prisma:
  id, userId FK, dayOfWeek (0=Sunday to 6=Saturday), startHour (0-23), endHour (0-23)
  @@unique([userId, dayOfWeek, startHour])

POST /users/me/availability — @UseGuards(JwtAuthGuard)
  Body: { slots: Array<{dayOfWeek, startHour, endHour}> }
  Upsert all slots (deleteMany existing + createMany new)

GET /users/:id/availability — public
  Return all availability slots for user

== SESSION BOOKING ==
POST /sessions/book — @UseGuards(JwtAuthGuard)
DTO: BookSessionDto { skillId: string, startTime: Date }

5 Validations (throw BadRequestException for each):
  1. Skill exists and isActive = true
  2. skill.userId !== req.user.id (cannot book own skill)
  3. startTime > new Date() (must be in the future)
  4. Student has sufficient balance: user.tokenBalance >= skill.priceCoins
  5. Overlap check: no existing session for mentor OR student with status IN [PENDING, ACTIVE]
where startTime < newEndTime AND endTime > newStartTime

Overlap check SQL (use prisma.$queryRaw):
  SELECT id FROM sessions
  WHERE (mentor_id = $mentorId OR student_id = $studentId)
  AND status IN ('PENDING', 'ACTIVE')
  AND start_time < $newEndTime
  AND end_time > $newStartTime

Calculate endTime:
  BEGINNER skill: startTime + 30 minutes
  INTERMEDIATE skill: startTime + 45 minutes
```

```
  EXPERT skill: startTime + 60 minutes

Atomic booking (prisma.$transaction):
  1. LedgerService.lockEscrow(sessionId, studentId, skill.priceCoins)
  2. prisma.session.create({ data: { mentorId, studentId, skillId, startTime, endTime, status: PENDING
} })
  3. NotificationsService.create(mentorId, 'SESSION_BOOKED', 'New session booked for [skill.title]',
sessionId)

Return: created session with skill and student info

== FRONTEND MARKETPLACE PAGE ==
MarketplacePage (/marketplace):

SearchBar:
  Controlled input, debounced 300ms (use custom useDebounce hook)
  Sync search term to URL params (?q=react)
  On change: invalidate and refetch marketplace query

FilterSidebar (collapsible on mobile):
  Category: multi-select checkboxes (fetch unique categories from GET /skills/categories)
  Level: radio buttons (BEGINNER / INTERMEDIATE / EXPERT / All)
  Price range: dual-handle slider (Shadcn Slider), min 0 max 500
  Available Now: toggle switch (checks Redis via API)
  Sort by: select dropdown
  Apply Filters button + Reset button

SkillCard component:
  User avatar with online presence dot (green if online)
  Skill title + category badge
  Level badge (color-coded: green/yellow/red)
  Rating stars + reputation score
  Teaching streak with fire icon
  Price in SkillCoins with coin icon
  'Book Now' button → opens BookingModal

TanStack Query setup:
```

```
  useQuery({ queryKey: ['marketplace', filters], queryFn: () => searchSkills(filters), staleTime:
30_000, placeholderData: keepPreviousData })

BookingModal:
  Shows: skill title, mentor name, price
  Slot picker: calendar + time slots based on mentor availability
  Current balance display with warning if balance < price
  Escrow notice: 'Tokens will be held in escrow until session completion'
  Confirm button: disabled if insufficient balance
  On confirm: POST /sessions/book → on success: toast + navigate /sessions/:id

SkillDetailPage (/marketplace/:skillId):
  Hero section: skill title, category, level, price
  Mentor profile card: avatar, name, reputation, streak, verified badge
  GitHub section: if githubId, show link to profile (fetch repos via GitHub API client-side)
  LinkedIn section: if linkedinId, show job title
  Description section: full markdown-rendered description
  Reviews section: last 10 reviews with star ratings and comments
  Booking section: sticky sidebar with BookingModal trigger

== CRITICAL RULES ==
- GIN index on search_vector is MANDATORY — without it, FTS is a full table scan
- Booking is ATOMIC: escrow lock + session create in same prisma.$transaction
- Overlap check MUST run before escrow lock
- NEVER expose passwordHash in any skill or user response
- Soft delete only for skills — hard delete breaks session history
- TanStack Query staleTime: 30_000 on all marketplace queries to prevent excessive refetching
- URL params must reflect filter state (shareable URLs)
```

Phase 4 — Live Classroom (WebRTC + Monaco + Excalidraw)

This phase builds the core value-delivery feature — the live classroom where the actual skill exchange happens. LiveKit abstracts away raw WebRTC complexity and provides data channels for syncing the collaboration tools. Monaco Editor and Excalidraw are synced via LiveKit's data channel, avoiding the need for a separate WebSocket sync server. The entire LiveClassroomPage is lazy-loaded to keep the initial bundle small.

Key Deliverables

- LiveKitService: room token generation with time-gating and participant restriction
- Session lifecycle API: join-token, complete, dispute endpoints
- Redis tracking for join state (both participants joined → ACTIVE)
- LiveClassroomPage: full-screen layout with video panel + tabbed collaboration panel
- Monaco Editor synced via LiveKit data channel (debounced 200ms)
- Excalidraw whiteboard synced via LiveKit data channel (debounced 300ms)
- Countdown timer broadcast by mentor via data channel
- Dual-confirmation completion flow → escrow release → RatingModal

Implementation Prompt — Phase 4

PHASE 4 PROMPT — LIVE CLASSROOM

Build the live classroom feature for SkillBarter using LiveKit for WebRTC.

== LIVEKIT SERVICE ==
Install: @livekit/server-sdk

LiveKitService.generateRoomToken(roomName: string, participantIdentity: string, participantName: string): string
  const at = new AccessToken(LIVEKIT_API_KEY, LIVEKIT_API_SECRET, { identity: participantIdentity, name: participantName })
  at.addGrant({ roomJoin: true, room: roomName, canPublish: true, canSubscribe: true, canPublishData: true })
  return at.toJwt()

== SESSION ENDPOINTS ==
POST /sessions/:id/join-token — @UseGuards(JwtAuthGuard)
  5 Validations:
  1. Session exists: prisma.session.findUniqueOrThrow
  2. Participant check: session.mentorId === userId OR session.studentId === userId, else ForbiddenException
  3. Not too early: new Date() >= session.startTime - 10 minutes, else BadRequestException('Room not open yet')
  4. Not too late: new Date() <= session.endTime + 5 minutes, else BadRequestException('Session has ended')

```
  5. Status check: session.status IN [PENDING, ACTIVE], else BadRequestException

  Generate roomId if null: roomId = `session-${sessionId}-${Date.now()}`
  Update session: if livekit_room_id is null, set it now
  Track join in Redis: SET session:{sessionId}:joined:{userId} '1' EX 7200
  Check if both joined: GET session:{sessionId}:joined:{mentorId} AND GET
session:{sessionId}:joined:{studentId}
  If both joined AND session.status === PENDING: update session status to ACTIVE, emit WebSocket
'session.started'
  Generate token: LiveKitService.generateRoomToken(roomId, userId, user.name)
  Return: { token: string, roomId: string, liveKitUrl: LIVEKIT_URL }

POST /sessions/:id/complete — @UseGuards(JwtAuthGuard)
  Validate: session exists, user is participant, status === ACTIVE
  Determine which flag to set: isMentor ? { mentorCompleted: true } : { studentCompleted: true }
  Update session with flag
  Re-fetch session to check both flags:
  If session.mentorCompleted && session.studentCompleted:
    prisma.$transaction:
      LedgerService.releaseEscrow(sessionId)
      prisma.session.update({ status: COMPLETED })
    emit WebSocket 'session.completed' to both participants
    NotificationsService.create for both users
  Else:
    emit WebSocket 'session.completion_pending' to the OTHER participant
  Return: { bothCompleted: boolean, session }

POST /sessions/:id/dispute — @UseGuards(JwtAuthGuard)
  Validate: session exists, user is participant, status === ACTIVE
  DTO: DisputeDto { reason: string (min 20 chars) }
  prisma.$transaction:
    LedgerService.freezeEscrow(sessionId)
    prisma.session.update({ status: DISPUTED })
    prisma.dispute.create({ sessionId, raisedBy: userId, reason, status: OPEN })
  NotificationsService.create for admin user
  NotificationsService.create for other participant
  emit WebSocket 'session.disputed' to both participants
```

```
  Return: created dispute

== FRONTEND LIVE CLASSROOM PAGE ==
Install: @livekit/components-react, @livekit/components-styles, @monaco-editor/react,
@excalidraw/excalidraw

Lazy load in router:
  const LiveClassroomPage = React.lazy(() => import('./pages/LiveClassroomPage'))
  Wrap in <Suspense fallback={<ClassroomLoadingScreen />}>

LiveClassroomPage layout (full viewport, no scrolling):
  Header bar (h-12): session title + countdown timer + Mark Complete button + Raise Dispute
button
  Main area (flex row, remaining height):
    Left panel (w-1/2): LiveKit video
    Right panel (w-1/2): Tabs (Code Editor | Whiteboard)

Data fetching:
  useQuery({ queryKey: ['session', 'join-token', sessionId], queryFn: () => getJoinToken(sessionId) })
  Destructure: { token, roomId, liveKitUrl }

LiveKit integration:
  <LiveKitRoom serverUrl={liveKitUrl} token={token} connect={true} audio={true} video={true}>
    <VideoConference />
    <DataChannelSync />  {/* custom component */}
  </LiveKitRoom>

DataChannelSync component (inside LiveKitRoom context):
  const { send: sendCode } = useDataChannel('code-sync', (msg) => {
setEditorContent(decode(msg.payload)) })
  const { send: sendWhiteboard } = useDataChannel('whiteboard-sync', (msg) => {
setExcalidrawState(JSON.parse(decode(msg.payload))) })
  const { send: sendTimer } = useDataChannel('timer-sync', (msg) => {
setTimeRemaining(parseInt(decode(msg.payload))) })

Monaco Editor tab:
  <Editor height='100%' language='javascript' value={editorContent}
```

```
    onChange={(value) => { debouncedSendCode(encode(value)) }}  // debounce 200ms
    options={{ minimap: { enabled: false }, fontSize: 14 }} />
  Track sequence numbers to prevent echo: if message came from self, ignore

Excalidraw tab:
  <Excalidraw initialData={excalidrawState}
    onChange={(elements, state) => { debouncedSendWhiteboard(encode(JSON.stringify({elements,
state})) }}  // debounce 300ms
  />

Countdown timer:
  isMentor: useEffect with setInterval 1000ms, calculate remaining = endTime - now,
sendTimer(remaining.toString()), clearInterval on unmount
  isStudent: receive timer-sync messages, update timeRemaining state
  Display: format as MM:SS, red when < 5 minutes
  When timer hits 0: auto-call handleMarkComplete()

handleMarkComplete():
  POST /sessions/:id/complete
  If response.bothCompleted === true: show RatingModal
  Else: show toast 'Waiting for other participant to confirm'

RatingModal:
  Star rating component (1-5, interactive)
  Comment textarea (optional)
  Submit: POST /reviews { sessionId, rating, comment }
  On success: navigate('/dashboard') + success toast

handleRaiseDispute():
  Confirmation dialog: 'Are you sure? Tokens will be frozen pending admin review.'
  Reason textarea (min 20 chars)
  POST /sessions/:id/dispute
  On success: navigate('/sessions') + warning toast

== CRITICAL RULES ==
- LiveKit token generation is TIME-GATED on the backend — never generate tokens client-side
- BOTH participants must mark complete — one-sided completion does nothing to escrow
```

- Data channel messages need sequence numbers for Monaco to prevent out-of-order application
- Lazy load the ENTIRE LiveClassroomPage — it imports Monaco + Excalidraw which are large
- Handle LiveKit disconnection gracefully: show reconnecting overlay, attempt reconnect 3 times
- Timer is broadcast by MENTOR only — student only receives, never broadcasts timer
- Dispute reason minimum 20 characters — enforce on both frontend and backend

Phase 5 — Async Bounty Board

The Bounty Board is SkillBarter's async learning layer — Stack Overflow but with video answers. It allows users to earn tokens without scheduling live sessions, directly solving the cold-start problem for new users who have knowledge but no scheduled sessions yet. Video answers are uploaded directly to Cloudinary/S3 via presigned URLs — never routed through the NestJS server.

Key Deliverables

- Presigned URL upload endpoint (Cloudinary signed URL or S3 presigned PUT, expires 5 min)
- Bounty CRUD with immediate escrow lock on creation
- Answer submission with video URL validation
- Accept answer flow with escrow release and 5% tax
- Auto-expiry cron job (@Cron EVERY_HOUR) — idempotent, refunds escrow to poster
- Frontend: BountyBoardPage, Post Bounty modal, BountyDetailPage with video player

Implementation Prompt — Phase 5

PHASE 5 PROMPT — ASYNC BOUNTY BOARD

Build the Bounty Board feature for SkillBarter — async video Q&A with token rewards.

== UPLOAD MODULE ==
Install: @aws-sdk/client-s3, @aws-sdk/s3-request-presigner (or cloudinary SDK)

GET /upload/presigned-url — @UseGuards(JwtAuthGuard)
Query params: fileType (e.g. 'video/mp4'), fileName

For S3:

```
  const key = `bounty-answers/${userId}/${Date.now()}-${fileName}`
  const command = new PutObjectCommand({ Bucket: S3_BUCKET, Key: key, ContentType: fileType
})
  const uploadUrl = await getSignedUrl(s3Client, command, { expiresIn: 300 })  // 5 minutes
```

```
  const fileUrl = `https://${S3_BUCKET}.s3.${S3_REGION}.amazonaws.com/${key}`
  Return: { uploadUrl, fileUrl }

For Cloudinary:
  const timestamp = Math.round(Date.now() / 1000)
  const signature = cloudinary.utils.api_sign_request({ timestamp, folder: 'bounty-answers' },
CLOUDINARY_SECRET)
  Return: { uploadUrl: cloudinary upload endpoint, fileUrl: constructed URL, signature, timestamp,
apiKey }

== BOUNTIES MODULE ==
POST /bounties — @UseGuards(JwtAuthGuard)
  DTO: CreateBountyDto { title: string (max 100), description: string (min 20), category: string,
rewardCoins: number (min 5) }
  Validate: user.tokenBalance >= rewardCoins, else BadRequestException('Insufficient balance')
  expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)  // 7 days from now
  prisma.$transaction:
    bounty = prisma.bounty.create({ data: { posterId, title, description, category, rewardCoins, status:
OPEN, expiresAt } })
    LedgerService.lockBountyEscrow(bounty.id, posterId, rewardCoins)
  Return: created bounty

GET /bounties — public
  Query params: status (default OPEN), category, sortBy (newest|reward|expiring_soon), page,
limit
  Include: poster (name, reputationScore), _count: { answers: true }
  Filter: WHERE expiresAt > now() for OPEN bounties
  Sort: newest → createdAt DESC, reward → rewardCoins DESC, expiring_soon → expiresAt ASC
  Return paginated results

GET /bounties/:id — public
  Include: poster, answers (with responder info), escrow status
  Return full bounty detail

POST /bounties/:id/answers — @UseGuards(JwtAuthGuard)
  Validate:
    1. Bounty exists and status === OPEN
```

```
    2. bounty.posterId !== userId (cannot answer own bounty)
    3. User has not already submitted an answer for this bounty
  Validate videoUrl: must start with allowed domain (S3 bucket URL or res.cloudinary.com)
  prisma.$transaction:
    prisma.bountyAnswer.create({ data: { bountyId, responderId: userId, videoUrl } })
    prisma.bounty.update({ where: {id: bountyId}, data: { status: ANSWERED } })
  NotificationsService.create(bounty.posterId, 'BOUNTY_ANSWERED', 'Your bounty has a new video
answer', bountyId)
  Return: created answer

POST /bounties/:id/answers/:answerId/accept — @UseGuards(JwtAuthGuard)
  Validate:
    1. Bounty exists
    2. bounty.posterId === userId (only poster can accept)
    3. bounty.status !== CLOSED (cannot accept twice)
    4. Answer exists and belongs to this bounty
  prisma.$transaction:
    prisma.bountyAnswer.update({ where: {id: answerId}, data: { isAccepted: true } })
    prisma.bounty.update({ where: {id: bountyId}, data: { status: CLOSED } })
    LedgerService.releaseBountyEscrow(bountyId, answer.responderId)
  NotificationsService.create(answer.responderId, 'BOUNTY_ACCEPTED', 'Your answer was
accepted! Tokens transferred.', bountyId)
  Return: updated bounty

== BOUNTY EXPIRY CRON JOB ==
Install: @nestjs/schedule

@Cron(CronExpression.EVERY_HOUR)
async expireOldBounties(): Promise<void>
  const expiredBounties = await prisma.bounty.findMany({
    where: { status: { in: [OPEN, ANSWERED] }, expiresAt: { lt: new Date() } },
    include: { escrow: true }
  })
  for (const bounty of expiredBounties) {
    if (!bounty.escrow || bounty.escrow.status !== LOCKED) continue  // IDEMPOTENT: skip if
already processed
    await prisma.$transaction(async (tx) => {
```

```
    await tx.bounty.update({ where: {id: bounty.id}, data: { status: EXPIRED } })
    await LedgerService.refundBountyEscrow(bounty.id)  // full refund to poster, no tax
  })
  await NotificationsService.create(bounty.posterId, 'BOUNTY_EXPIRED', 'Your bounty expired.
Tokens refunded.', bounty.id)
 }
 // Log: logger.log(`Expired ${expiredBounties.length} bounties`)

== FRONTEND BOUNTY BOARD ==
BountyBoardPage (/bounties):
 Filter bar: status tabs (Open | Answered | Closed | All), category filter, sort dropdown
 'Post Bounty' button (top right, requires auth)
 Bounty card grid (2 cols desktop, 1 col mobile):
  Title + description preview (truncated 100 chars)
  Category badge
  Reward badge: gold background, coin icon, reward amount
  Expiry countdown: 'Expires in 3d 4h' (red if < 24h)
  Answer count badge
  'View & Answer' button

Post Bounty Modal:
 Title input, description textarea, category select, reward input
 Balance check: show current balance, warning if reward > 50% of balance
 Escrow notice: 'Reward will be locked until accepted or expired'
 Submit: POST /bounties → on success: invalidate bounties query + success toast

BountyDetailPage (/bounties/:id):
 Full title + description
 Poster info + reward + expiry
 Status badge
 Submit Video Answer section (only if OPEN and not own bounty and authenticated):
  File input: accept='video/*', max 2 minutes (validate duration client-side)
  Upload flow:
   1. GET /upload/presigned-url?fileType=video/mp4&fileName=answer.mp4
   2. PUT uploadUrl with file (show progress bar)
   3. On upload complete: POST /bounties/:id/answers { videoUrl: fileUrl }
  Show upload progress with Shadcn Progress component
```

Answers list:
Each answer: responder avatar + name + submission date
HTML5 video player: <video src={answer.videoUrl} controls preload='metadata' />
'Accept Answer' button: only visible to poster, only on OPEN/ANSWERED bounties, disabled after any acceptance
Accepted badge: green checkmark on accepted answer

== CRITICAL RULES ==
- Upload goes DIRECTLY to Cloudinary/S3 — NEVER through NestJS (file size, memory)
- Presigned URL expires in 5 minutes — generate fresh on each upload attempt
- Only ONE answer can be accepted per bounty — enforce on backend
- Cron job is IDEMPOTENT — check escrow.status === LOCKED before processing
- Validate video URL domain on backend — only allow your S3 bucket or Cloudinary domain
- refundBountyEscrow has NO tax — full refund to poster on expiry
- releaseBountyEscrow HAS 5% tax — same as session escrow release

Phase 6 — Notifications + Real-time (WebSocket)

Real-time presence and notifications transform SkillBarter from a static booking platform into a live community. Redis Pub/Sub decouples the WebSocket gateway from application services, enabling horizontal scaling. Online status is the key UX feature driving 'Available Now' filtering in the marketplace. The heartbeat pattern with Redis TTL is simpler and more reliable than tracking disconnect events.

Key Deliverables

- NestJS WebSocket Gateway (Socket.IO, namespace /ws) with JWT authentication
- Redis Pub/Sub with two separate ioredis clients (pub and sub — cannot share)
- Online status tracking: Redis TTL-based with 20s client heartbeat
- DB-persisted notifications with real-time delivery via WebSocket
- Frontend: Zustand notification store, socket.ts, NotificationBell, online presence dots

Implementation Prompt — Phase 6

PHASE 6 PROMPT — NOTIFICATIONS + REAL-TIME WEBSOCKET

Build the real-time layer for SkillBarter using Socket.IO and Redis Pub/Sub.

== WEBSOCKET GATEWAY ==

```
Install: @nestjs/websockets, @nestjs/platform-socket.io, socket.io, ioredis

@WebSocketGateway({ namespace: '/ws', cors: { origin: FRONTEND_URL, credentials: true } })
export class WsGateway implements OnGatewayConnection, OnGatewayDisconnect

handleConnection(client: Socket):
  Extract JWT from client.handshake.auth.token
  Verify JWT: jwtService.verify(token, { secret: JWT_ACCESS_SECRET })
  If invalid: client.disconnect(true); return
  const userId = payload.sub
  client.data.userId = userId
  client.join(`user:${userId}`)  // join personal room
  await redisPub.set(`user:${userId}:online`, '1', 'EX', 30)  // 30 second TTL
  await redisPub.publish('presence', JSON.stringify({ userId, status: 'online' }))

handleDisconnect(client: Socket):
  const userId = client.data.userId
  if (!userId) return
  await redisPub.del(`user:${userId}:online`)
  await redisPub.publish('presence', JSON.stringify({ userId, status: 'offline' }))

@SubscribeMessage('heartbeat')
handleHeartbeat(client: Socket):
  const userId = client.data.userId
  await redisPub.expire(`user:${userId}:online`, 30)  // refresh TTL
  return { status: 'ok' }

emitToUser(userId: string, event: string, data: any):
  this.server.to(`user:${userId}`).emit(event, data)

== REDIS PUB/SUB SETUP ==
CRITICAL: Create TWO separate ioredis instances — you CANNOT publish and subscribe on the
same client
  const redisPub = new Redis(REDIS_URL)  // for publishing and regular commands
  const redisSub = new Redis(REDIS_URL)  // ONLY for subscribing

In WsGateway onModuleInit():
```

```
  await redisSub.subscribe('presence', 'notifications', 'session-events')
  redisSub.on('message', (channel, message) => {
    const data = JSON.parse(message)
    if (channel === 'presence') {
      this.server.emit(`user.${data.status}`, { userId: data.userId })
    }
    if (channel === 'notifications') {
      this.emitToUser(data.userId, 'notification.new', data.notification)
    }
    if (channel === 'session-events') {
      this.emitToUser(data.userId, data.event, data.payload)
    }
  })
```

== ALL WEBSOCKET EVENTS ==

Define as constants:

session.booked — emitted to mentor when student books
session.started — emitted to both when both join LiveKit
session.completed — emitted to both when both mark complete
session.completion_pending — emitted to other participant when one marks complete
session.disputed — emitted to both when dispute raised
bounty.answered — emitted to poster when answer submitted
bounty.accepted — emitted to responder when answer accepted
notification.new — emitted to user when any notification created
user.online — broadcast when user comes online
user.offline — broadcast when user goes offline

== NOTIFICATIONS SERVICE ==

```
async create(userId: string, type: string, message: string, referenceId?: string):
Promise<Notification>
  ALWAYS create DB record FIRST:
    const notification = await prisma.notification.create({ data: { userId, type, message, referenceId }
})
  THEN emit via WebSocket (publish to Redis channel):
    await redisPub.publish('notifications', JSON.stringify({ userId, notification }))
  Return: notification
```

```
GET /notifications/me — @UseGuards(JwtAuthGuard)
  Query params: page (default 1), limit (default 20), unreadOnly (boolean)
  WHERE userId = me, if unreadOnly: AND isRead = false
  ORDER BY createdAt DESC
  Also return: unreadCount (COUNT WHERE isRead = false)
  Return: { data, total, unreadCount }

POST /notifications/:id/read — @UseGuards(JwtAuthGuard)
  Validate: notification.userId === req.user.id
  Update: isRead = true

POST /notifications/read-all — @UseGuards(JwtAuthGuard)
  Update all: WHERE userId = me AND isRead = false → isRead = true

== ONLINE STATUS ENDPOINT ==
GET /users/online-status — @UseGuards(JwtAuthGuard)
  Query params: ids (comma-separated userIds)
  For each userId: check Redis key user:{userId}:online
  Return: { [userId]: boolean }
  Limit: max 50 IDs per request

== FRONTEND SOCKET SETUP ==
src/lib/socket.ts:
  import { io, Socket } from 'socket.io-client'
  let socket: Socket | null = null
  export function connectSocket(accessToken: string): Socket {
    socket = io(VITE_API_URL, {
      path: '/ws',
      auth: { token: accessToken },
      reconnection: true,
      reconnectionAttempts: 5,
      reconnectionDelay: 1000,
    })
    // Heartbeat: send every 20 seconds
    const heartbeatInterval = setInterval(() => { socket?.emit('heartbeat') }, 20_000)
    socket.on('disconnect', () => clearInterval(heartbeatInterval))
    return socket
```

```
}
export function disconnectSocket(): void { socket?.disconnect(); socket = null }
export function getSocket(): Socket | null { return socket }
```

Zustand auth store: call connectSocket(accessToken) in setAuth, disconnectSocket() in clearAuth

Zustand NotificationStore (src/stores/notificationStore.ts):

```
interface NotificationState {
  notifications: Notification[]
  unreadCount: number
  addNotification(n: Notification): void  // prepend to array, increment unreadCount
  markRead(id: string): void
  markAllRead(): void
  setUnreadCount(count: number): void
  setNotifications(notifications: Notification[], unreadCount: number): void
}
```

Zustand OnlineUsersStore (src/stores/onlineUsersStore.ts):

```
interface OnlineUsersState {
  onlineUsers: Set<string>
  setOnline(userId: string): void
  setOffline(userId: string): void
  isOnline(userId: string): boolean
}
```

Socket event listeners in App.tsx (inside useEffect on mount):

```
const socket = getSocket()
if (!socket) return
socket.on('notification.new', (notification) => {
  addNotification(notification)
  toast({ title: notification.message, variant: 'default' })
})
socket.on('session.completed', () => {
  queryClient.invalidateQueries({ queryKey: ['sessions'] })
  queryClient.invalidateQueries({ queryKey: ['wallet'] })
  toast({ title: 'Session completed! Tokens transferred.', variant: 'success' })
})
```

```
  socket.on('user.online', ({ userId }) => setOnline(userId))
  socket.on('user.offline', ({ userId }) => setOffline(userId))
  return () => { socket.off('notification.new'); socket.off('session.completed'); ... }

NotificationBell component (in navbar):
  Bell icon (Lucide BellIcon)
  Unread badge: red circle with count (hidden if 0)
  Dropdown (Shadcn Popover): last 5 notifications, each with type icon + message + time ago
  'Mark all read' button → POST /notifications/read-all → setUnreadCount(0)
  'View all' link → /notifications
  On mount: fetch GET /notifications/me?limit=5 → setNotifications

Online presence dot on user avatars:
  <div className='relative'>
    <Avatar />
    {isOnline(userId) && <span className='absolute bottom-0 right-0 w-3 h-3 bg-green-500
rounded-full border-2 border-white' />}
  </div>
  Fallback: if userId not in onlineUsersStore, poll GET /users/online-status?ids=userId every 60s

== CRITICAL RULES ==
- TWO Redis clients are MANDATORY — ioredis throws if you subscribe and publish on same client
- Heartbeat is MANDATORY — without it, Redis TTL expires and users appear offline after 30s
- ALWAYS save notification to DB BEFORE emitting WebSocket event
- JWT validation in handleConnection is MANDATORY — reject unauthenticated connections
immediately
- Socket rooms follow pattern user:{userId} — never use socket.id for targeting
- Reconnection config is important — mobile users frequently disconnect
```

Phase 7 — Admin Panel + Dispute Resolution

The admin panel is the trust and safety backbone of SkillBarter. Dispute resolution is the most sensitive operation in the platform — it directly moves tokens between users and must be fully auditable. All admin routes are protected by role-based guards. The SPLIT decision math must be documented in code comments. Every admin action is auditable via the Transaction table.

Key Deliverables

- AdminModule with ADMIN role guard on all routes (no exceptions)
- Dispute resolution API with three decision types: RELEASE_TO_MENTOR, REFUND_TO_STUDENT, SPLIT
- User management: ban/unban with cascade cancellation of pending sessions
- Platform statistics: totalUsers, sessions, escrows, disputes, token circulation, tax collected
- Admin wallet: tax revenue tracking with daily breakdown
- Frontend: AdminPanel with sidebar, disputes table + resolve modal, users table, stats charts

Implementation Prompt — Phase 7

```
PHASE 7 PROMPT — ADMIN PANEL + DISPUTE RESOLUTION

Build the admin panel for SkillBarter. ALL routes in this module require @Roles('ADMIN') +
@UseGuards(JwtAuthGuard, RolesGuard).

== DISPUTE RESOLUTION ==
GET /admin/disputes
  Query params: status (OPEN|RESOLVED|all), page, limit
  Include: session (with mentor, student, skill), escrow, raisedByUser
  ORDER BY: OPEN first, then createdAt DESC

GET /admin/disputes/:id
  Full detail: dispute + session + mentor profile + student profile + escrow + all session transactions

POST /admin/disputes/:id/resolve
  DTO: ResolveDisputeDto { decision: DisputeDecision, adminNotes: string (min 20 chars) }
  Validate: dispute exists, status === OPEN, adminNotes.length >= 20
  Find: escrow by sessionId, session details

  Decision logic (document math in code comments):

  RELEASE_TO_MENTOR:
    // Mentor wins: release escrow with 5% tax as normal completion
    await LedgerService.releaseEscrow(sessionId)  // handles tax internally
    await prisma.session.update({ where: {id: sessionId}, data: { status: COMPLETED } })
```

```
  REFUND_TO_STUDENT:
    // Student wins: full refund, no tax (admin absorbs loss)
    await LedgerService.refundEscrow(sessionId)  // full amount back to student
    await prisma.session.update({ where: {id: sessionId}, data: { status: CANCELLED } })

  SPLIT:
    // 50/50 split with tax on mentor's half only
    // half = Math.floor(escrow.amount / 2)
    // tax = Math.floor(half * 0.05)
    // mentorReceives = half - tax
    // studentReceives = escrow.amount - half  (no tax on student's portion)
    // adminReceives = tax
    const half = Math.floor(escrow.amount / 2)
    const tax = Math.floor(half * 0.05)
    const mentorReceives = half - tax
    const studentReceives = escrow.amount - half
    await prisma.$transaction(async (tx) => {
      await tx.user.update({ where: {id: mentorId}, data: { tokenBalance: { increment: mentorReceives
} } })
      await tx.user.update({ where: {id: studentId}, data: { tokenBalance: { increment:
studentReceives } } })
      await tx.user.update({ where: {id: adminId}, data: { tokenBalance: { increment: tax } } })
      await tx.transaction.create({ data: { toUserId: mentorId, amount: mentorReceives, type:
ESCROW_RELEASE, referenceId: sessionId } })
      await tx.transaction.create({ data: { toUserId: studentId, amount: studentReceives, type:
ESCROW_RELEASE, referenceId: sessionId } })
      await tx.transaction.create({ data: { toUserId: adminId, amount: tax, type: TAX, referenceId:
sessionId } })
      await tx.escrow.update({ where: {sessionId}, data: { status: RELEASED, resolvedAt: new Date() }
})
      await tx.session.update({ where: {id: sessionId}, data: { status: COMPLETED } })
    })

  Always after any decision:
    await prisma.dispute.update({ where: {id: disputeId}, data: { status: RESOLVED, adminNotes,
decision, resolvedAt: new Date() } })
    await NotificationsService.create(mentorId, 'DISPUTE_RESOLVED', `Dispute resolved:
```

```
${decision}`, disputeId)
    await NotificationsService.create(studentId, 'DISPUTE_RESOLVED', `Dispute resolved:
${decision}`, disputeId)

== USER MANAGEMENT ==
GET /admin/users
  Query params: search (email or name ILIKE), role, isBanned, page, limit
  Include: _count { sessions: true, skills: true }
  Return paginated users (NEVER include passwordHash)

GET /admin/users/:id
  Full profile: user + last 10 transactions + last 5 sessions + last 5 bounties + reviews received

POST /admin/users/:id/ban
  DTO: BanUserDto { reason: string (min 10 chars) }
  Validate: cannot ban another ADMIN
  prisma.$transaction:
    prisma.user.update({ where: {id}, data: { isBanned: true } })
    // Cancel all PENDING sessions and refund escrows:
    const pendingSessions = await prisma.session.findMany({ where: { OR: [{mentorId: id},
{studentId: id}], status: PENDING } })
    for (const session of pendingSessions) {
      await LedgerService.refundEscrow(session.id)
      await prisma.session.update({ where: {id: session.id}, data: { status: CANCELLED } })
    }
  NotificationsService.create(id, 'ACCOUNT_BANNED', `Your account has been banned. Reason:
${reason}`)

POST /admin/users/:id/unban
  prisma.user.update({ where: {id}, data: { isBanned: false } })
  NotificationsService.create(id, 'ACCOUNT_UNBANNED', 'Your account has been reinstated.')

== PLATFORM STATISTICS ==
GET /admin/stats
  Return object with:
  totalUsers: COUNT users WHERE role = USER
  totalSessions: COUNT sessions
```

```
  completedSessions: COUNT sessions WHERE status = COMPLETED
  activeEscrows: COUNT escrows WHERE status = LOCKED
  openDisputes: COUNT disputes WHERE status = OPEN
  totalTokensInCirculation: SUM tokenBalance WHERE role = USER (excludes admin)
  totalTaxCollected: SUM amount FROM transactions WHERE type = TAX
  dailyActiveUsers: approximate from Redis (count user:*:online keys) — use SCAN not KEYS
  topSkillCategories: GROUP BY category ORDER BY COUNT DESC LIMIT 5
  recentSignups: COUNT users WHERE createdAt > NOW() - INTERVAL '7 days' GROUP BY
DATE(createdAt)

GET /admin/wallet
  adminUser = prisma.user.findFirst({ where: { role: ADMIN } })
  currentBalance: adminUser.tokenBalance
  totalTaxCollected: SUM transactions WHERE type = TAX AND toUserId = adminUser.id
  taxThisMonth: SUM WHERE type = TAX AND toUserId = adminUser.id AND createdAt > start of
month
  taxThisWeek: SUM WHERE type = TAX AND toUserId = adminUser.id AND createdAt > start of
week
  taxByDay: last 30 days, GROUP BY DATE(createdAt), SUM amount WHERE type = TAX

GET /admin/bounties — paginated list of all bounties with poster info
DELETE /admin/bounties/:id
  Validate: bounty.status !== CLOSED (cannot delete closed bounties)
  prisma.$transaction:
    if escrow.status === LOCKED: LedgerService.refundBountyEscrow(bountyId)
    prisma.bounty.update({ data: { status: EXPIRED } })
  NotificationsService.create(bounty.posterId, 'BOUNTY_REMOVED', 'Your bounty was removed by
admin. Tokens refunded.')

== FRONTEND ADMIN PANEL ==
AdminLayout: fixed left sidebar (240px) + main content area
Sidebar items: Dashboard (BarChart icon), Disputes (AlertTriangle + unread badge), Users (Users
icon), Bounties (Video icon), Wallet (Wallet icon)
All admin routes: /admin, /admin/disputes, /admin/users, /admin/bounties, /admin/wallet
Protect with: <ProtectedRoute requiredRole='ADMIN' />

AdminDashboard (/admin):
```

6 stats cards (2 rows of 3): Total Users, Total Sessions, Completed Sessions, Active Escrows, Open Disputes, Tokens in Circulation
Line chart: daily signups last 7 days (Recharts LineChart)
Bar chart: sessions per day last 7 days (Recharts BarChart)
Pie chart: top 5 skill categories (Recharts PieChart)

AdminDisputesPage (/admin/disputes):
Status filter tabs: All | Open | Resolved
DataTable columns: ID (truncated), Session, Mentor, Student, Amount, Raised By, Status, Created, Actions
'Resolve' button opens DisputeDetailModal:
Full dispute info: reason, session details, escrow amount
Transaction history for this session
Decision radio group: Release to Mentor | Refund to Student | Split 50/50
Admin Notes textarea (required, min 20 chars, show char count)
Confirm button: disabled until decision selected and notes >= 20 chars
On submit: POST /admin/disputes/:id/resolve → close modal + invalidate disputes query + success toast

AdminUsersPage (/admin/users):
Search input (debounced 300ms)
Filter: role, isBanned toggle
DataTable: Avatar, Name, Email, Balance, Sessions, Role, Status (Active/Banned), Actions
Ban action: opens BanModal with reason input (min 10 chars)
Unban action: confirmation dialog

AdminWalletPage (/admin/wallet):
Balance card: large number, 'Admin Treasury'
3 stat cards: Total Tax Collected, This Month, This Week
Area chart: tax revenue last 30 days (Recharts AreaChart)

== CRITICAL RULES ==
- EVERY admin route MUST have @Roles('ADMIN') + @UseGuards(JwtAuthGuard, RolesGuard) — no exceptions
- SPLIT math MUST be documented in code comments (half, tax, mentorReceives, studentReceives)
- Ban MUST cancel all pending sessions and refund escrows in same transaction
- adminNotes is REQUIRED on dispute resolution — minimum 20 characters

- Cannot ban another ADMIN — check role before banning
- All admin actions are auditable via the Transaction table — never bypass LedgerService
- Use Redis SCAN not KEYS for daily active users — KEYS blocks the Redis event loop

Phase 8 — DevOps: Docker + Redis + CI/CD + Production

This phase wraps the entire platform in production-grade infrastructure. Multi-stage Docker builds minimize image sizes. GitHub Actions automates the full deploy pipeline from lint to production. Production hardening — Helmet, rate limiting, CORS, Pino logging, connection pooling — is non-negotiable before going live. The health check endpoint must pass before any deployment proceeds.

Key Deliverables

- Multi-stage Dockerfiles for backend (node:20-alpine) and frontend (nginx:alpine)
- docker-compose.yml with all 5 services: postgres, redis, backend, frontend, nginx
- Nginx reverse proxy config with WebSocket upgrade support and SSL
- GitHub Actions CI/CD: lint → test → build → push → deploy via SSH
- NestJS production hardening: Helmet, ThrottlerModule, ValidationPipe, Pino logger
- Jest unit tests for LedgerService (7 test cases covering all financial operations)
- Health check endpoints via @nestjs/terminus

Implementation Prompt — Phase 8

PHASE 8 PROMPT — DEVOPS: DOCKER + CI/CD + PRODUCTION HARDENING

Wrap SkillBarter in production-grade infrastructure.

```
== BACKEND DOCKERFILE (apps/backend/Dockerfile) ==
# Stage 1: Builder
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npx prisma generate
RUN npm run build

# Stage 2: Production
```

```
FROM node:20-alpine AS production
WORKDIR /app
RUN addgroup -g 1001 -S nodejs && adduser -S nestjs -u 1001
COPY --from=builder --chown=nestjs:nodejs /app/node_modules ./node_modules
COPY --from=builder --chown=nestjs:nodejs /app/dist ./dist
COPY --from=builder --chown=nestjs:nodejs /app/prisma ./prisma
USER nestjs
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/api/health || exit 1
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/main.js"]

== FRONTEND DOCKERFILE (apps/frontend/Dockerfile) ==
# Stage 1: Builder
FROM node:20-alpine AS builder
WORKDIR /app
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production (nginx)
FROM nginx:alpine AS production
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s CMD wget --spider http://localhost || exit 1
CMD ["nginx", "-g", "daemon off;"]

Frontend nginx.conf (SPA config):
  server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
    # Cache static assets for 1 year
```

```
    location ~* \.(js|css|png|jpg|ico|woff2)$ {
      expires 1y;
      add_header Cache-Control 'public, immutable';
    }
    # SPA fallback
    location / {
      try_files $uri $uri/ /index.html;
    }
  }

== DOCKER COMPOSE (docker-compose.yml) ==
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment: { POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD }
    volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
      interval: 10s, timeout: 5s, retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru --requirepass
${REDIS_PASSWORD}
    volumes: [redis_data:/data]
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 10s, timeout: 5s, retries: 5

  backend:
    build: { context: ./apps/backend, dockerfile: Dockerfile }
    env_file: .env
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    restart: unless-stopped
```

```
  frontend:
    build:
      context: ./apps/frontend
      dockerfile: Dockerfile
      args: { VITE_API_URL: ${VITE_API_URL} }
    depends_on: [backend]
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports: ['80:80', '443:443']
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/ssl:/etc/nginx/ssl:ro
    depends_on: [backend, frontend]
    restart: unless-stopped

volumes: { postgres_data: {}, redis_data: {} }

== NGINX REVERSE PROXY (nginx/nginx.conf) ==
events { worker_connections 1024; }
http {
  gzip on;
  gzip_types text/plain application/json application/javascript text/css;

  upstream backend { server backend:3000; }
  upstream frontend { server frontend:80; }

  # HTTP → HTTPS redirect
  server {
    listen 80;
    return 301 https://$host$request_uri;
  }

  server {
    listen 443 ssl;
```

```
    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    # API routes
    location /api/ {
      proxy_pass http://backend;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }

    # WebSocket upgrade (CRITICAL for Socket.IO)
    location /ws/ {
      proxy_pass http://backend;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
      proxy_set_header Connection 'upgrade';
      proxy_set_header Host $host;
      proxy_read_timeout 86400;  # 24 hours — keep WS connections alive
    }

    # Frontend SPA
    location / {
      proxy_pass http://frontend;
      proxy_set_header Host $host;
    }
  }
}

== NESTJS PRODUCTION HARDENING (apps/backend/src/main.ts) ==
Install: nestjs-pino, pino-http, pino-pretty, @nestjs/terminus, helmet, compression,
@nestjs/throttler

async function bootstrap() {
  const app = await NestFactory.create(AppModule, { bufferLogs: true })

  // Pino logger
```

```
  app.useLogger(app.get(Logger))

  // Security
  app.use(helmet({ contentSecurityPolicy: { directives: { defaultSrc: ["'self'"], ... } } }))
  app.use(compression())

  // CORS — only allow frontend URL
  app.enableCors({ origin: process.env.FRONTEND_URL, credentials: true, methods:
['GET','POST','PUT','DELETE','PATCH'] })

  // Global prefix
  app.setGlobalPrefix('api')

  // Validation
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform:
true, transformOptions: { enableImplicitConversion: true } }))

  await app.listen(3000)
}

ThrottlerModule in AppModule:
  ThrottlerModule.forRoot([{ ttl: 60_000, limit: 100 }])  // 100 req/min global
  On auth endpoints: @Throttle({ default: { ttl: 60_000, limit: 5 } })  // 5/min on login/register

Pino config in AppModule:
  LoggerModule.forRoot({
    pinoHttp: {
      level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
      transport: process.env.NODE_ENV !== 'production' ? { target: 'pino-pretty' } : undefined,
      redact: ['req.headers.authorization', 'req.headers.cookie', 'req.body.password'],
    }
  })

== HEALTH CONTROLLER ==
@Controller('health')
export class HealthController {
  constructor(private health: HealthCheckService, private prismaHealth: PrismaHealthIndicator,
```

```
private redis: MicroserviceHealthIndicator) {}

  @Get()
  @HealthCheck()
  check() {
    return this.health.check([
      () => this.prismaHealth.pingCheck('database'),
      () => this.redis.pingCheck('redis', { transport: Transport.REDIS, options: { host: 'redis', port: 6379
} })
    ])
  }
}

== GITHUB ACTIONS CI/CD (.github/workflows/deploy.yml) ==
name: Deploy SkillBarter
on: push: branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4 with: { node-version: '20' }
      - run: npm ci
      - run: npm run type-check  # tsc --noEmit
      - run: npm run lint
      - run: npm run test  # Jest unit tests

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3 with: { registry: ghcr.io, username: ${{ github.actor }}, password:
${{ secrets.GITHUB_TOKEN }} }
      - uses: docker/build-push-action@v5 with: { context: ./apps/backend, push: true, tags:
ghcr.io/${{ github.repository }}/backend:latest }
```

```
    - uses: docker/build-push-action@v5 with: { context: ./apps/frontend, push: true, tags:
ghcr.io/${{ github.repository }}/frontend:latest, build-args: VITE_API_URL=${{ secrets.VITE_API_URL
}} }

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1 with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            cd /opt/skillbarter
            docker-compose pull
            docker-compose up -d
            docker image prune -f
```

== JEST UNIT TESTS FOR LEDGER SERVICE ==
Create: apps/backend/src/ledger/ledger.service.spec.ts

Mock prisma.$transaction to execute the callback with a mock tx object
7 test cases:

1. credit() — should increment balance and create EARN transaction
2. debit() — should decrement balance and create SPEND transaction
3. debit() — should throw BadRequestException when balance insufficient
4. lockEscrow() — should decrement student balance and create LOCKED escrow
5. releaseEscrow() — should credit mentor with amount minus 5% tax (Math.floor)
6. releaseEscrow() — tax math: 100 coins → tax=5, mentor=95, verify Math.floor(100*0.05)===5
7. applyNoShowPenalty() — should not go below 0 (mentor with 3 coins → penalty = 3 not 10)

== .env.example (complete) ==

```
# Database
DATABASE_URL=postgresql://user:password@localhost:5432/skillbarter
# Redis
REDIS_URL=redis://:password@localhost:6379
REDIS_PASSWORD=your_redis_password
```

```
# JWT
JWT_ACCESS_SECRET=your_access_secret_min_32_chars
JWT_REFRESH_SECRET=your_refresh_secret_min_32_chars
# OAuth
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
GITHUB_CALLBACK_URL=https://yourdomain.com/api/auth/github/callback
LINKEDIN_CLIENT_ID=
LINKEDIN_CLIENT_SECRET=
LINKEDIN_CALLBACK_URL=https://yourdomain.com/api/auth/linkedin/callback
# LiveKit
LIVEKIT_API_KEY=
LIVEKIT_API_SECRET=
LIVEKIT_URL=wss://your-livekit-instance.livekit.cloud
# Storage
CLOUDINARY_URL=cloudinary://api_key:api_secret@cloud_name
# App
FRONTEND_URL=https://yourdomain.com
ADMIN_EMAIL=admin@skillbarter.dev
NODE_ENV=production

== CRITICAL RULES ==
- Multi-stage builds are MANDATORY — single-stage images are too large for production
- CMD must run 'prisma migrate deploy' before starting the app — never skip migrations
- NEVER commit .env — add to .gitignore immediately
- Redis maxmemory-policy allkeys-lru PREVENTS out-of-memory crashes
- Pino MUST redact authorization headers and cookies — never log tokens
- Rate limit auth endpoints to 5/min — brute force protection
- Health check must pass BEFORE deploy job runs — add health check step
- WebSocket proxy_read_timeout 86400 is MANDATORY — without it Nginx kills WS connections
after 60s
```

APPENDIX — QUICK REFERENCE

Token Math Cheat Sheet

TABLE 13 — TOKEN MATH CHEAT SHEET

Operation	Formula	Example	Notes
Tax calculation	Math.floor(amount × 0.05)	100 coins → tax = 5	ALWAYS floor — never round or ceil
Mentor receives	amount − tax	100 − 5 = 95 coins	Net after tax deduction
No-show penalty	Math.min(10, mentorBalance)	Balance=3 → penalty=3 (not 10)	Prevents negative balance
Split (dispute)	Math.floor(amount / 2)	100 → 50 each (before tax)	Tax applied to mentor's half only
Split mentor net	half − Math.floor(half × 0.05)	50 − 2 = 48 coins	Tax on mentor portion only
Split student net	amount − half	100 − 50 = 50 coins	No tax on student's portion in split
Signup bonus	50 coins (flat)	New user → +50 coins	EARN transaction required
OAuth bonus	50 coins per provider	GitHub link → +50 coins	Idempotent — check ID was null first

Session Status State Machine

TABLE 14 — SESSION STATUS STATE MACHINE

From State	To State	Trigger	Side Effects
PENDING	ACTIVE	Both participants join LiveKit room (tracked in Redis)	Emit 'session.started' WebSocket event
ACTIVE	COMPLETED	Both participants click 'Mark Complete'	LedgerService.releaseEscrow() → 5% tax → mentor credited → streak++
ACTIVE	DISPUTED	Either participant raises dispute	LedgerService.freezeEscrow() → admin notified → tokens frozen
PENDING	CANCELLED	Mentor no-show (<1hr) or admin action	LedgerService.refundEscrow() → full refund to student → no-show penalty applied
DISPUTED	COMPLETED	Admin resolves:	LedgerService handles per

		RELEASE_TO_MENTOR or SPLIT	decision type
DISPUTED	CANCELLED	Admin resolves: REFUND_TO_STUDENT	LedgerService.refundEscrow() → full refund

Escrow Status State Machine

TABLE 15 — ESCROW STATUS STATE MACHINE

From State	To State	Trigger	Who Receives Tokens
LOCKED	RELEASED	Session completed (both confirm) OR bounty answer accepted	Mentor (session) or Responder (bounty) — minus 5% tax to admin
LOCKED	DISPUTED	Dispute raised by either participant	Nobody — tokens frozen pending admin review
LOCKED	REFUNDED	Session cancelled, no-show, or bounty expired	Student (session) or Poster (bounty) — full amount, no tax
DISPUTED	RELEASED	Admin resolves: RELEASE_TO_MENTOR or SPLIT	Per admin decision — see dispute resolution logic
DISPUTED	REFUNDED	Admin resolves: REFUND_TO_STUDENT	Student — full amount, no tax

API Authentication Summary

TABLE 16 — API AUTHENTICATION SUMMARY

Endpoint Group	Auth Required	Role Required	Notes
POST /auth/*	No	—	Register, login, refresh, OAuth callbacks
GET /marketplace/search	No	—	Public discovery — no auth needed
GET /skills/:id	No	—	Public skill detail page
GET /bounties	No	—	Public bounty board
GET /bounties/:id	No	—	Public bounty detail (posting answer requires auth)
POST /skills	Yes (JWT)	USER	Create skill listing
PUT /skills/:id	Yes (JWT)	USER (owner only)	Edit own skill —

			ownership guard
POST /sessions/book	Yes (JWT)	USER	Book a session — triggers escrow lock
POST /sessions/:id/complete	Yes (JWT)	USER (participant)	Mark session complete
POST /sessions/:id/dispute	Yes (JWT)	USER (participant)	Raise dispute — freezes escrow
POST /bounties	Yes (JWT)	USER	Post bounty — triggers escrow lock
POST /bounties/:id/answers	Yes (JWT)	USER	Submit video answer
GET /wallet/*	Yes (JWT)	USER	Balance, summary, transaction history
GET /notifications/me	Yes (JWT)	USER	Personal notifications
GET /upload/presigned-url	Yes (JWT)	USER	Get presigned URL for video upload
GET /admin/*	Yes (JWT)	ADMIN	All admin read endpoints
POST /admin/*	Yes (JWT)	ADMIN	All admin write endpoints — fully audited

Environment Variables Reference

TABLE 17 — ENVIRONMENT VARIABLES REFERENCE (18 VARIABLES)

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string
REDIS_URL	Yes	Redis connection string with password
REDIS_PASSWORD	Yes	Redis auth password (also in REDIS_URL)
JWT_ACCESS_SECRET	Yes	Min 32 chars — signs 15-minute access tokens
JWT_REFRESH_SECRET	Yes	Min 32 chars — signs 7-day refresh tokens
GITHUB_CLIENT_ID	Yes	GitHub OAuth App client ID
GITHUB_CLIENT_SECRET	Yes	GitHub OAuth App client secret
GITHUB_CALLBACK_URL	Yes	Must match GitHub OAuth App settings exactly

LINKEDIN_CLIENT_ID	Yes	LinkedIn OAuth App client ID
LINKEDIN_CLIENT_SECRET	Yes	LinkedIn OAuth App client secret
LINKEDIN_CALLBACK_URL	Yes	Must match LinkedIn OAuth App settings exactly
LIVEKIT_API_KEY	Yes	LiveKit project API key
LIVEKIT_API_SECRET	Yes	LiveKit project API secret
LIVEKIT_URL	Yes	LiveKit server URL (wss://...)
CLOUDINARY_URL	Yes (or S3 vars)	Cloudinary connection string
FRONTEND_URL	Yes	Frontend origin for CORS (https://yourdomain.com)
ADMIN_EMAIL	Yes	Email for seeded admin user
NODE_ENV	Yes	production | development

> FINAL REMINDER: Execute phases in order. Phase 1 (schema + auth) is the foundation. Phase 2 (LedgerService) is the most critical — a bug here means real token loss. Test LedgerService thoroughly with the 7 Jest unit tests before building any feature that calls it.

SkillBarter Implementation Guide — Department of Computer Engineering, University of Mumbai, Academic Year 2025–2026. Submitted by Adit Halde (337), Kartik Hulmukh (338), Vivek Humbe (339). Supervised by Prof. Kaajal Sharma.